

Lista de Exercícios — Programação Orientada a Objetos com Python

Esta lista foi estruturada para cobrir praticamente todos os conceitos importantes de POO em Python:

- Classes e objetos
 - Métodos
 - Construtores
 - Encapsulamento
 - Herança
 - Polimorfismo
 - Classes abstratas
 - Métodos especiais (`__str__`, `__len__`, etc.)
 - Composição
 - Associação
 - Properties
 - Decorators
 - Classes estáticas
 - Tratamento de exceções
 - Organização de projetos
 - Boas práticas
-

Exercício 1 — Classe Pessoa

Crie uma classe `Pessoa` com:

- `nome`
- `idade`
- `profissão`

Métodos:

- `apresentar()`
- `fazer_aniversario()`

Objetivo

Treinar:

- criação de classes
- atributos
- métodos
- `__init__`

Dica

Pesquise sobre:

- `self`
 - método construtor `__init__`
 - diferença entre atributo e método
-

Exercício 2 — Sistema Bancário Simples

Crie uma classe `ContaBancaria`.

Atributos:

- titular
- saldo
- número da conta

Métodos:

- `depositar()`
- `levantar()`
- `transferir()`

Regras:

- não permitir saldo negativo
- validar valores negativos

Objetivo

Treinar:

- encapsulamento
- validação

- regras de negócio

Dica

Pesquise sobre:

- atributos privados `_saldo` e `__saldo`
 - `raise ValueError`
 - validações dentro de métodos
-

Exercício 3 — Sistema de Biblioteca

Crie:

- classe `Livro`
- classe `Biblioteca`

A biblioteca deve:

- adicionar livros
- listar livros
- emprestar livros
- devolver livros

Objetivo

Treinar:

- composição
- listas de objetos
- relacionamento entre classes

Dica

Pesquise:

- listas contendo objetos
 - `for objeto in lista`
 - composição em POO
-

Exercício 4 — Herança de Veículos

Crie:

- classe `Veiculo`
- subclasses:
 - `Carro`
 - `Moto`
 - `Caminhao`

Todos devem possuir:

- `marca`
- `modelo`
- `ligar()`

Cada subclasse deve implementar:

- `descricao()`

Objetivo

Treinar:

- herança
- sobrescrita de métodos
- polimorfismo

Dica

Pesquise:

- `super()`
- `override` de métodos
- diferença entre herança e composição

Exercício 5 — Sistema Escolar

Crie:

- Pessoa
- Aluno
- Professor

Professor:

- disciplina
- salário

Aluno:

- número do estudante
- notas

Métodos:

- calcular média
- verificar aprovação

Objetivo

Treinar:

- reutilização de código
- herança múltipla (opcional)
- organização de classes

Dica

Pesquise:

- reutilização com herança
- métodos compartilhados
- boas práticas de modelagem

Exercício 6 — Carrinho de Compras

Crie:

- Produto
- Carrinho

Carrinho deve:

- adicionar produto
- remover produto
- calcular total
- listar itens

Objetivo

Treinar:

- composição
- manipulação de objetos
- agregação

Dica

Pesquise:

- `sum()`
 - listas de objetos
 - relacionamento "tem um"
-

Exercício 7 — Encapsulamento Real

Crie uma classe `Funcionario`.

Requisitos:

- salário privado
- getter e setter
- impedir salário negativo

Objetivo

Treinar:

- encapsulamento real
- `@property`
- setters

Dica

Pesquise:

- `@property`
 - `@atributo.setter`
 - convenções Python
-

Exercício 8 — Classe Abstrata

Crie uma classe abstrata `Animal`.

Método abstrato:

- `emitir_som()`

Subclasses:

- Cachorro
- Gato
- Leão

Objetivo

Treinar:

- abstração
- classes abstratas

Dica

Pesquise:

- módulo `abc`
 - `ABC`
 - `@abstractmethod`
-

Exercício 9 — Sistema de Funcionários

Crie:

- `Funcionario`
- `Gerente`
- `Desenvolvedor`

Cada um deve calcular bônus de forma diferente.

Objetivo

Treinar:

- polimorfismo
- sobrescrita
- métodos especializados

Dica

Pesquise:

- polimorfismo em Python
- sobrescrita de métodos
- chamada dinâmica

Exercício 10 — Métodos Especiais

Crie uma classe `Produto`.

Implemente:

- `__str__`
- `__repr__`
- `__len__`
- `__eq__`

Objetivo

Treinar:

- dunder methods
- personalização de objetos

Dica

Pesquise:

- magic methods
 - representação de objetos
 - operadores personalizados
-

Exercício 11 — Sistema de Login

Crie:

- `Usuario`
- autenticação
- alteração de senha

Regras:

- senha mínima
- senha privada
- autenticação correta

Objetivo

Treinar:

- encapsulamento
- segurança básica
- validações

Dica

Pesquise:

- hash de senha com `hashlib`
 - encapsulamento
 - validação de dados
-

Exercício 12 — Sistema de Inventário

Crie:

- Produto
- Estoque

Funcionalidades:

- entrada de stock
- saída de stock
- alerta de stock baixo

Objetivo

Treinar:

- manipulação de estados
- lógica empresarial

Dica

Pesquise:

- composição
 - validação de quantidades
 - separação de responsabilidades
-

Exercício 13 — Sistema de Rede de TI

(Baseado em ambiente real)

Crie:

- Servidor
- Switch
- Firewall
- Cliente

Todos devem herdar de:

- Equipamento

Funcionalidades:

- ligar/desligar
- verificar status
- mostrar IP

Objetivo

Treinar:

- herança
- arquitetura orientada a objetos
- modelagem de infraestrutura

Dica

Pesquise:

- classes base
 - reutilização de atributos
 - abstração
-

Exercício 14 — Backup Manager

Crie:

- Backup
- BackupLocal
- BackupCloud

Métodos:

- iniciar backup
- verificar integridade
- restaurar backup

Objetivo

Treinar:

- polimorfismo
- herança
- especialização

Dica

Pesquise:

- override
 - classes filhas
 - interfaces abstratas
-

Exercício 15 — Mini ERP

Crie:

- Cliente
- Produto
- Pedido
- Fatura
- Funcionário

Regras:

- pedido contém vários produtos
- calcular impostos
- emitir fatura

Objetivo

Treinar:

- modelagem completa
- múltiplas classes
- organização de projeto

Dica

Pesquise:

- separação em ficheiros
- arquitetura MVC simples

- módulos Python
-

Exercício 16 — Sistema de Monitoramento

Crie:

- `Sensor`
- `SensorTemperatura`
- `SensorRede`
- `SensorCPU`

Todos devem:

- coletar dados
- gerar alerta

Objetivo

Treinar:

- polimorfismo
- abstração
- especialização

Dica

Pesquise:

- padrões de projeto simples
 - herança abstrata
 - tratamento de eventos
-

Exercício 17 — Projeto Final Completo

Desenvolva um sistema completo à sua escolha:

Sugestões:

- Sistema de escola
- Sistema hospitalar
- Sistema de CCTV
- Sistema bancário
- Gestão de backups
- Gestão de tickets de TI

Requisitos obrigatórios

Usar:

- herança
- polimorfismo
- encapsulamento
- composição
- classes abstratas
- tratamento de exceções
- módulos separados
- persistência em ficheiro JSON

Dica

Pesquise:

- `json`
- organização de projetos Python
- `try/except`
- estrutura profissional de aplicações

Desafio Extra — Nível Profissional

Transforme um dos projetos acima em:

- API com Django ou Flask
- Interface gráfica com Tkinter
- Terminal interativo
- Sistema com PostgreSQL

Dica

Pesquise:

- ORM
 - arquitetura em camadas
 - DTO
 - Repository Pattern
 - SOLID
-

Ordem Recomendada de Estudo

1. Classes e objetos
 2. Métodos
 3. Encapsulamento
 4. Herança
 5. Polimorfismo
 6. Abstração
 7. Composição
 8. Métodos especiais
 9. Organização de projeto
 10. SOLID
-

Conceitos Que Deve Dominar no Final

Se conseguir resolver quase todos os exercícios, deve dominar:

- Modelagem orientada a objetos
- Estruturação de sistemas
- Reutilização de código
- Separação de responsabilidades
- Arquitetura básica de software
- Criação de sistemas reais em Python